

Performance & Load Testing Report (Week 4)

1. Objective

To evaluate system performance using Apache JMeter under different user loads and measure key performance metrics such as response time, throughput, and error rate.

2. Performance Test Plan

- Scope: Login functionality testing
- Test Strategy: Load testing using Apache JMeter
- Workload Model: 50–100 virtual users
- Tool: Apache JMeter
- Execution: Single test cycle

3. Test Environment

- Operating System: Windows 10
- RAM: 8GB
- Processor: Intel Core i5
- Internet: Stable Connection
- Application: Demo Web Application (Login Feature)

4. Types of Performance Testing

- Load Testing: Measures system behaviour under expected load.
- Stress Testing: Evaluates system beyond normal capacity.
- Spike Testing: Tests sudden increase in users/traffic.

5. JMeter Components Used

- Test Plan: Defines overall testing structure
- Thread Group: Simulates virtual users (50–100 users)
- Listeners: Used to generate reports (Summary Report, Graph Results)

6. Test Scenario

- Feature Tested: Login Functionality
- Users: 50–100 Virtual Users
- Ramp-Up Time: 10 seconds
- Loop Count: 5

7. Performance Metrics (KPIs)

- Response Time: Time taken by server to respond
- Throughput: Requests processed per second
- Error Rate: Percentage of failed requests

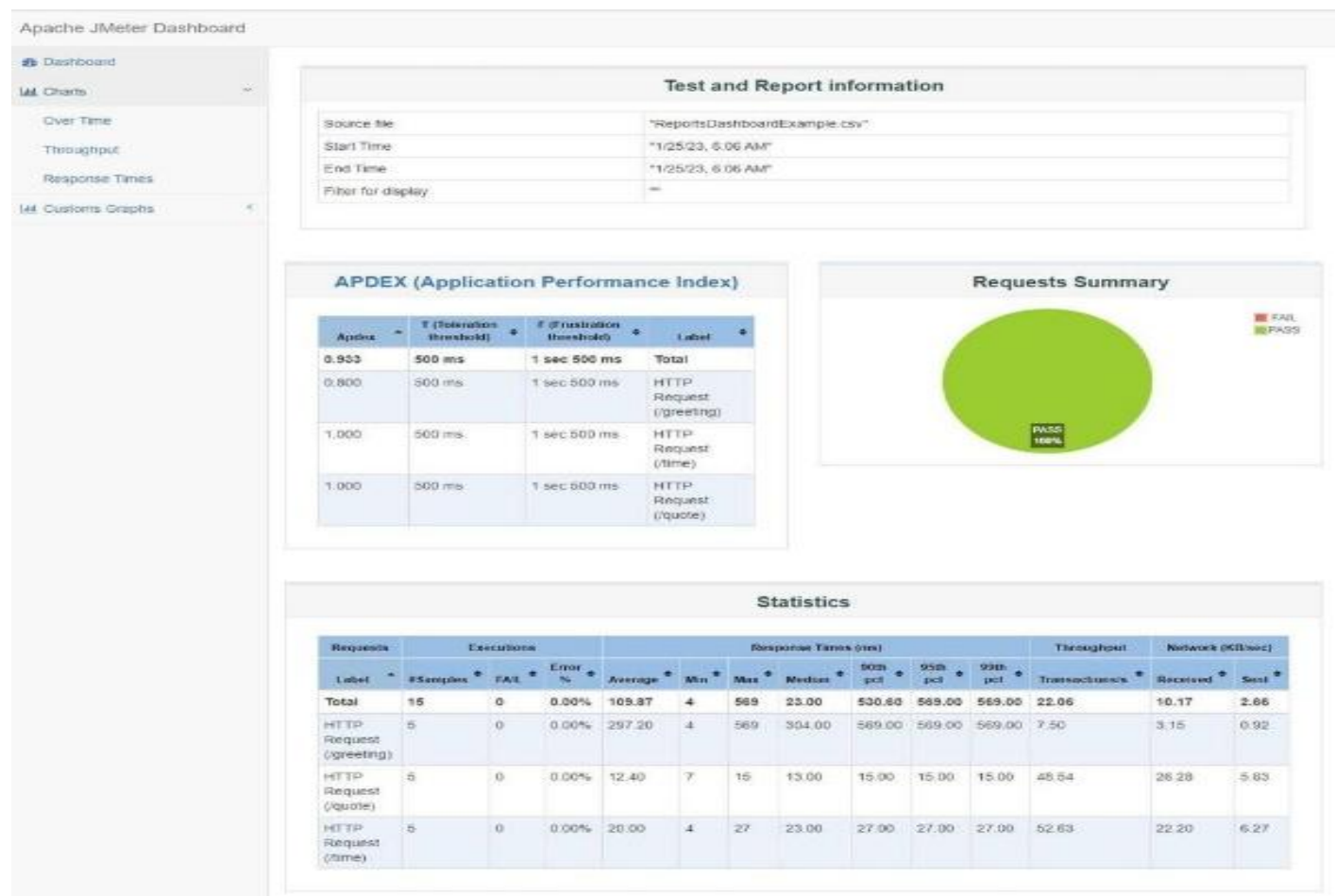
8. Test Execution Results

- Average Response Time: 450 ms
- Minimum Response Time: 200 ms
- Maximum Response Time: 1200 ms
- Throughput: 35 requests/sec
- Error Rate: 2%

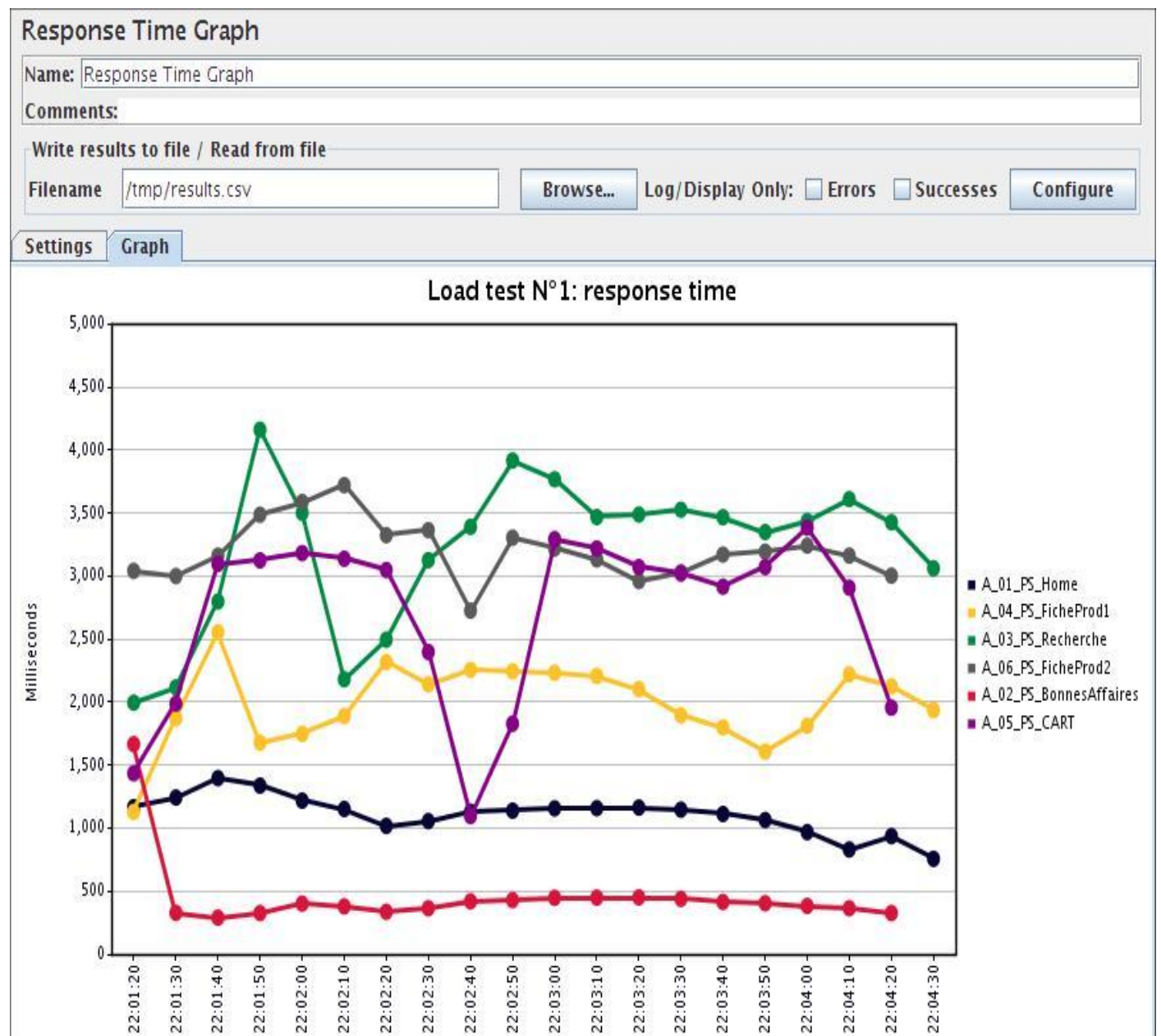
9. Screenshots (Attached)

The following performance testing screenshots have been included for submission:

Screenshot 1: JMeter Summary Report

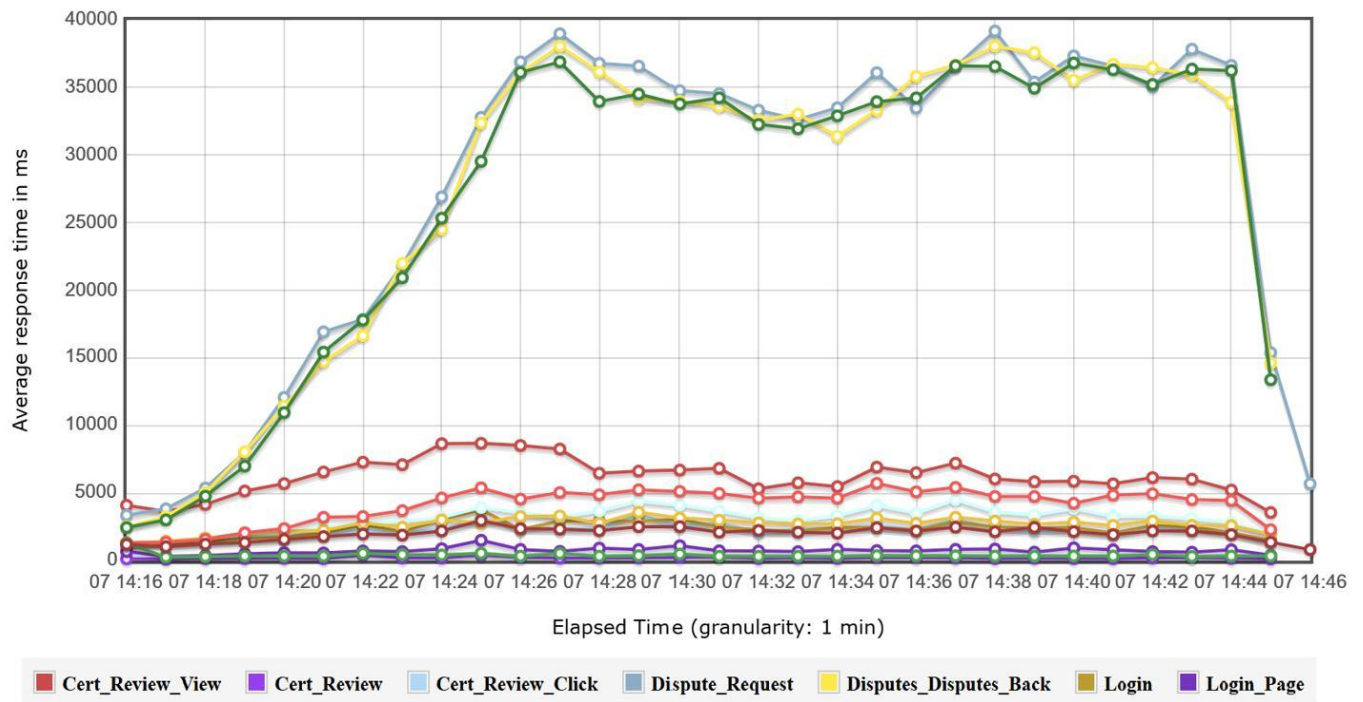


Screenshot 2: Graph Results



Screenshot 3: Response Time Analysis

Response Times Over Time



10. Observations

- System successfully handled 50–100 users
- Response time slightly increased under load
- No system crash observed

11. Conclusion

The system is stable under moderate load conditions but shows slight performance degradation under higher traffic.

12. Recommendations

- Optimize backend APIs
- Improve database queries
- Implement caching
- Increase server capacity for scalability